boost camp

The Bioinformatics Working Environment

Session 01 — Build the stack: terminal, conda, environments and tools









Build the stack, one layer at a time

Six layers, built from the laptop up. Each rests on the one below.

Layer	In plain words	You will use
1 Laptop	Your own computer	Windows 10/11 or macOS 11+
2 Terminal	Where you type commands	Ubuntu (WSL) or Mac Terminal
3 Package manager	Installs tools and what they need	Miniforge: conda and mamba
4 Environments	Named toolboxes you switch on	protein_modeling, meta_env
5 Tools	The programs that do the science	PyMOL, BLAST, Biopython and more
6 Companions	Utilities, file history, an editor	apt/Homebrew, Git, GitHub, VS Code

Four questions this session answers

Slides marked Q1 to Q4 answer them; the modules are in the Setup PDF.

The question	Short answer	Module
Q1 Install WSL Ubuntu on Windows?	`wsl --install`, then work in Ubuntu	1
Q2 Set up a Mac for bioinformatics?	Terminal, chip check, folders, Homebrew	2, 3
Q3 Conda on Windows, Ubuntu, Mac?	Miniforge; on Windows, inside Ubuntu	3
Q4 PyMOL, GUI and command line?	One package does both: `pymol-open-source`	3

Which modules are mine?

Follow your computer's row. Allow about 10 GB free and 1.5 to 2 hours.

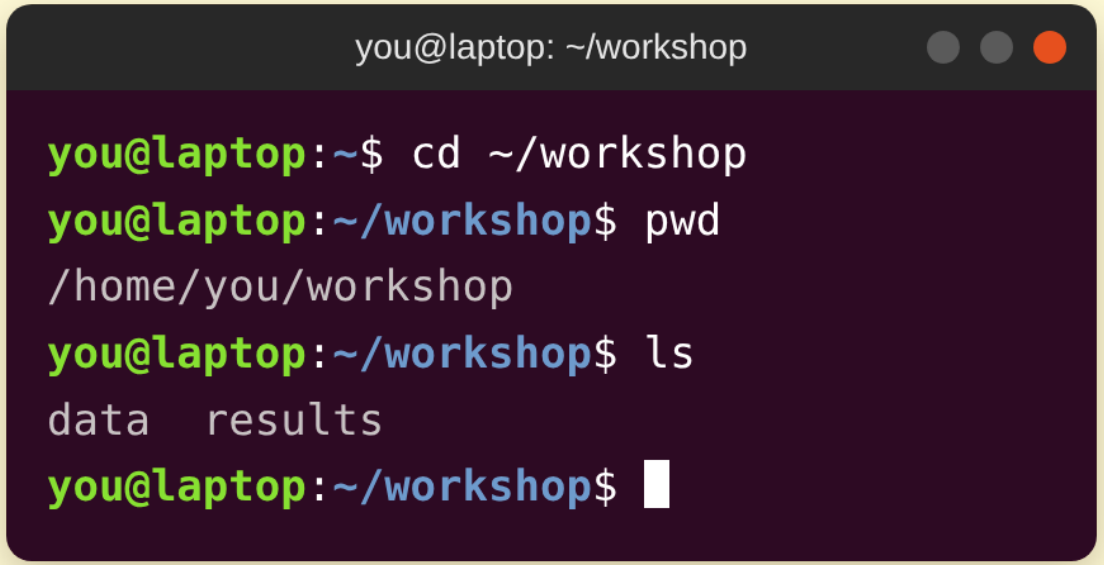
If you have…	Do this
Windows 10 or 11	Modules 1 → 3 (Ubuntu/WSL) → 4 → 5 → 6
A Mac, Apple silicon or Intel	Modules 2 → 3 (Mac) → 4 → 5 → 6
A metagenomics session	Add Module 7: +20 GB, +40 min, plus a database
No wish to publish online	Skip only the GitHub steps in 5 and 6



Layers 1–2: your laptop and a terminal

What is a terminal, and why use one?

- A **terminal** is a window where you type a command and press Enter.
- A **command** is one instruction: a program, then options and files.
- The **prompt**, such as `you@laptop:~$`, means "ready". Never type it.
- The **shell** reads your typing: bash on Ubuntu, zsh on a Mac.
- Most bioinformatics tools have no buttons, only commands.
- A typed command can be saved, rerun and shared. A click cannot.



Three terminals, and they are not interchangeable

Each step names its terminal. Use exactly that one.

Terminal	Its prompt	Use it for
Windows PowerShell	`PS C:\>`	Only `wsl` commands: install, check, update WSL
Ubuntu (WSL)	`you@laptop:~$`	Everything else, on Windows
Mac Terminal	`you@MacBook ~ %`	Everything, on a Mac

Golden rules, part 1: how to type

- **Type one line at a time.** Wait for the prompt to come back.
- **Never paste a block into an installer.** Pasted lines answer its questions by accident.
- **Passwords stay invisible:** no dots, no stars. Type it, press Enter.
- **Capitals count** on Linux: `cd Workshop` is not `cd workshop`.
- **Silence usually means success.** Many commands print nothing.
- **Frozen? Usually still downloading.** Give it a few minutes.

Q1: why Windows users work inside Ubuntu



- Workshop tools come from **bioconda**, a software store for Linux and macOS only.
- **Linux** is a free operating system; **Ubuntu** is a popular version of it.
- **WSL** (Windows Subsystem for Linux) runs real Ubuntu inside Windows.
- **WSLg**, part of WSL, shows Linux windows, such as PyMOL, on your desktop.
- Needs Windows 11, or Windows 10 build 19041+. PyMOL's window needs 19044+.
- Check your build under Settings → System → About.

Q1: install Ubuntu with WSL (Module 1)

```
# PowerShell, as administrator
wsl --install
# after the restart
wsl --list --verbose
```

- One command installs WSL and Ubuntu. Restart Windows when asked.
- Open **Ubuntu** from Start. The first launch takes about a minute.
- Choose a new Linux username and password, separate from Windows.
- The password stays invisible as you type. That is normal.
- The list must show Ubuntu at VERSION **2**.

Q1: set Ubuntu up, and work under /home

```
uname -m
sudo apt update
sudo apt install -y curl ca-certificates git
mkdir -p ~/workshop
cd ~/workshop
pwd
```

- `uname -m` prints your chip type, x86_64 or aarch64. Write it down.
- `sudo` means "as administrator"; **apt** is Ubuntu's own installer.
- `curl` downloads files from the web; `git` keeps file history.
- `~` is your home folder; `mkdir` makes a folder; `cd` enters it.
- `pwd` prints where you are. It must start with `/home/`.

Q2: get your Mac ready (Module 2)

```
sw_vers
uname -m
mkdir -p ~/workshop/data
mkdir -p ~/workshop/results
cd ~/workshop
pwd
```

- Open Terminal: Cmd+Space, type Terminal, press Enter.
- `sw_vers` shows your macOS; Miniforge needs macOS 11 or newer.
- `uname -m`: arm64 is Apple silicon, x86_64 is Intel.
- Originals go in `data`, everything you make in `results`.
- Leave macOS's own Python alone; conda brings its own.



Layers 3–4: packages and environments

What a package manager does

- A **package** is one program, bundled ready to install.
- Most programs need other packages to run: their **dependencies**.
- A **package manager** installs a tool plus every dependency, in versions that fit.
- **conda** is the package manager this workshop uses; **mamba** does the same, faster.
- **Miniforge** is the small official installer that gives you both.
- No hunting for installers, and no clicking through setup wizards.

Channels: where conda downloads from

A **channel** is an online store of ready-built packages.

Channel	What it holds	Order
conda-forge	General science: Python, PyMOL, pandas	Always first
bioconda	Bioinformatics tools: BLAST, SAMtools, FastQC	Always second

Q3: install conda on Windows, Ubuntu and Mac (Module 3)

Install Miniforge, then close the terminal, open a new one and type `conda --version`.

Your computer	Where you type	Miniforge installer
Windows 10 or 11	Ubuntu, never PowerShell	Linux `.sh`, run with `bash`
Ubuntu laptop	Terminal	Linux `.sh`, run with `bash`
Mac	Terminal	macOS `.pkg`, arm64 or x86_64
Old Windows, no PyMOL window	Miniforge Prompt	Windows `.exe`, for PyMOL only

What is an environment?

- An **environment** is a named box of software you switch on when needed.
- Each box has its own Python and its own tool versions.
- Tools in one box cannot break the tools in another.
- A broken box is deleted and rebuilt. Your data is not touched.
- **base** is conda's own box, not a workspace. Only mamba belongs there.
- This series uses **protein_modeling**; metagenomics adds **meta_env**.

Build protein_modeling (Module 3)

```
cd ~/workshop/Ologist_Workshop
ls
bash setup_protein_modeling_conda_envs.sh
```

- A **script** is a saved list of commands. This one builds and checks everything.
- Your instructor gives you the project folder. Keep it inside `~/workshop`.
- Success ends with `[OK] protein_modeling verified`.
- Running it again is safe: it adds anything missing and keeps the rest.
- No project folder? Type Module 3's `conda create` line instead.

Switch it on, every session

```
conda activate protein_modeling
python --version
conda info --envs
# at the end of the day:
conda deactivate
```

- **Activate** switches the box on, for this terminal window only.
- Your prompt now starts with `(protein_modeling)`.
- `python --version` reports 3.11.
- `conda info --envs` puts a `*` beside the active box.
- Install once. Activate at the start of every session.



Layer 5: the tools inside the environments

Where the tools fit: a map of later sessions

Everything runs in **protein_modeling**, except metagenomics, which uses **meta_env**.

Stage	Main tools
Metagenomics: reads to genes, which feed A	QC tools, assemblers, Bakta
A–C: match genes, extract and clean proteins	BLAST+, Python
D–E: files for the modelling websites	Python
F: check the downloaded models	PyMOL
Reference set: draw every structure	PyMOL

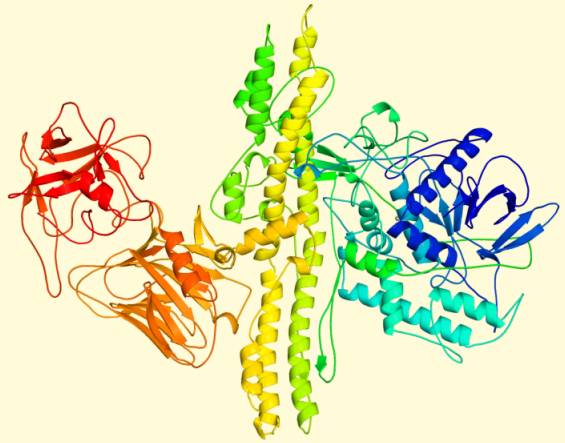
Python and the sequence tools

A **library** is ready-made code that scripts load with `import`.

Tool	What it does for you here
Python 3.11	Programming language of the workshop's helper scripts
Biopython	Reads, writes and aligns sequences and structures
pandas	Tables of data, like a spreadsheet in code
FastQC	Quality report on sequencing reads (a web page)
SeqKit	Counts and pulls out sequences in FASTA/FASTQ
BWA	Lines up short reads against a reference genome
SAMtools, BCFtools	Sort aligned reads (BAM); list sample differences (VCF)

Tools for search and structure

Tool	What it does for you here	Used in
BLAST+	Finds DNA sequences similar to yours	Program A
PyMOL	Draws and checks 3-D protein structures	Program F; figures



Botulinum neurotoxin A (PDB 3BTA), drawn by the workshop's own PyMOL script.

Q4: PyMOL, one package, three ways to use it



One free conda-forge package, `pymol-open-source`, is all three. It came with protein_modeling.

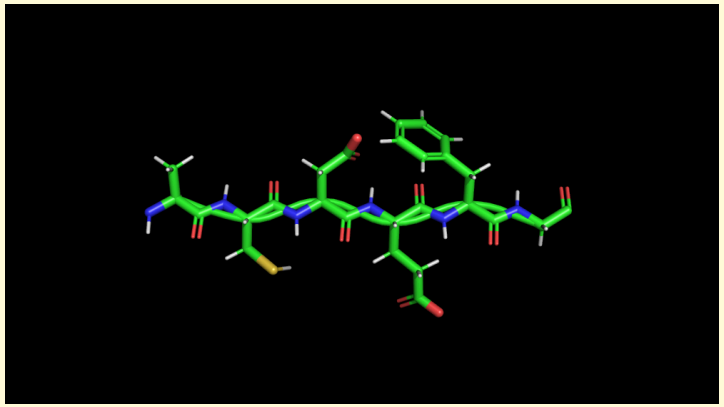
Way to use it	Command	Used for
Window (GUI)	`pymol`	Looking, by mouse or at `PyMOL>`
No window	`pymol -cq script.py`	Repeatable jobs and figures
Python library	`from pymol import cmd`	Inside program F and the drawing script

Q4: open PyMOL and prove the window draws



```
# In the terminal
conda activate protein_modeling
pymol --version
pymol
```

```
# At the PyMOL> prompt
fab ACDEFG, gui_test
show sticks, gui_test
orient
```



It works if you see this, and it turns as you drag. Keep the terminal open: closing it closes PyMOL.

Q4 on Windows: when the PyMOL window does not appear

On Windows, **WSLg** draws the window. It needs Windows 11, or Windows 10 build 19044+.

No window? Run these in **PowerShell**, then reopen Ubuntu:

```
wsl --update
wsl --shutdown
```

Windows too old for WSLg? A fallback, for viewing only:

- Install Windows Miniforge (`.exe`), then open **Miniforge Prompt**.
- Create `pymol_gui` from conda-forge, activate it, run `pymol`.
- It only views structures. Every other command stays in Ubuntu.

meta_env: the metagenomics toolbox (Module 7)

Only if your session includes metagenomics.

Job	Tools	In plain words
Read quality	FastQC, NanoStat	Reports on short and long reads
Read cleaning	fastp, fastplong	Trims adapters and poor-quality ends
Assembly	metaSPAdes, MetaFlye	Joins reads into longer stretches, contigs
Evaluation	metaQUAST, Bandage	Measures the assembly and draws its graph
Annotation	Bakta	Finds the genes and names them
Screening	AMRFinderPlus, ABRicate	Looks for resistance and virulence genes
Hybrid (optional)	OPERA-MS	Short + long reads; not in meta_env

Why meta_env stands apart, and how to build it

- The assemblers' dependencies would downgrade protein_modeling's packages.
- Bakta, AMRFinderPlus and ABRicate sit in two helper boxes; their libraries clash.
- The script links them into meta_env, so you only activate meta_env.
- Databases go to `~/workshop/databases`. Bakta's is 1.3 GB, ~4 GB unpacked.
- Download interrupted? Run the script again. It resumes where it stopped.

```
bash setup_metagenomics_conda_envs.sh
conda activate meta_env
```



Layer 6: the companions

apt and Homebrew: the system's own installers

- Installs small utilities into the machine, for every terminal.
- Ubuntu/WSL uses **apt**. Mac uses **Homebrew** (`brew`), from brew.sh.
- `tree -L 2 ~/workshop` draws your folders as an indented tree.
- `btop` shows CPU, memory and disk live: busy or stuck?
- `sudo` belongs to apt only. Never to conda or Homebrew.

```
# Ubuntu / WSL
sudo apt update
sudo apt install -y tree btop
# Mac
brew install tree btop
```

Git: a history of your work (Module 4)



- **Git** records each saved change to your files, on your own machine.
- A **repository** is a folder whose history Git tracks.
- A **commit** is one recorded snapshot, with a short message.
- Ubuntu/WSL: installed in Module 1. Mac: accept Apple's developer tools prompt.
- Use your own name and email; they appear in published history.

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

VS Code: your editor from here on (Module 6)

```
mkdir -p ~/workshop/git-practice
cd ~/workshop/git-practice
code .
# Mac: File → Open Folder instead
```

- A free editor for scripts and notes, with a terminal and Git buttons.
- Windows: install it on Windows itself, plus Microsoft's **WSL** extension.
- `WSL: Ubuntu`, bottom-left, means you are editing Linux files.
- Create `README.md`, then Source Control → Initialize Repository → stage → Commit.
- Practise in a new, empty folder, never a real research project.

GitHub and the data-safety rule (Modules 5 and 6)

- **GitHub** is a website that hosts a copy of your Git history.
- Optional: needed to publish or share, not to practise Git locally.
- **Turn on two-factor authentication.** Never show anyone your password.
- **Publish practice work as private.** Never force-push.
- **Never publish** patient data, unpublished data, passwords, `.env` files or tokens.
- **No large raw sequencing files** either. Check your institution's rules.



Ready, fix, ask

Golden rules, part 2: how to install

- **Reopen the terminal** after installing conda.
- **Never** `sudo conda`**.** sudo belongs to apt only.
- **Keep** `(base)` **clean:** no workshop packages in it.
- **On WSL, work under** `/home` (`~/workshop`), not `/mnt/c`.
- **Install once; activate every session.**
- **Official sources only** for every installer.

Readiness checkpoints: can you show these?

Layer	Proof it works
Ubuntu on Windows	VERSION 2 in PowerShell; `pwd` starts `/home/`
Mac	You know arm64 or x86_64; `cd ~/workshop` works
conda	A new terminal answers `conda --version`
protein_modeling	Prompt shows `(protein_modeling)`; Python reports 3.11
PyMOL	`pymol` opens a black canvas with `PyMOL>`
Git and VS Code	Your own name prints; a local commit exists
meta_env, if used	`conda info --envs` lists four workshop environments

When it fails: the terminal and conda

Symptom	What to do
`wsl` is not recognised	PowerShell as administrator; check Windows build
WSL install stalls at 0%	`wsl --install --web-download -d Ubuntu`
`conda: command not found`	New terminal; else `~/miniforge3/bin/conda init bash` (Mac: `zsh`)
conda in PowerShell, no tools	You installed Windows Miniforge. Use Ubuntu
Miniforge file is only a few KB	Failed download. Delete it, download again
`Permission denied`	Work in your home folder. Never `sudo conda`
Stuck at "Solving environment"	Wait several minutes. If it fails, rebuild

When it fails: tools, folders, PyMOL and Git

Symptom	What to do
`command not found` for a tool	Wrong environment. Read your prompt, then `conda activate`
`No such file or directory`	Wrong folder. `pwd`, `ls`, then `cd` to it
apt cannot find a package	`sudo apt update` first; htop if no btop
No PyMOL window on Windows	PowerShell: `wsl --update`, `wsl --shutdown`; reopen Ubuntu
Git missing inside VS Code	`git --version` in that system; restart VS Code
A tool says it has no database	Bakta: `--db ~/workshop/databases/db-light`; or rerun setup
Assembly stops with `Killed`	Out of memory. Smaller input, or the server

How to ask for help

Stop at the line that failed. Copy the **first** error before you try any fix.

Then bring these three things:

1. The exact command you typed
2. The full error message, not a summary of it
3. The output of `conda info --envs`

Never share passwords or tokens, not even with the instructor.

Next: 02 The Datasets

```
# before next session,
# in a fresh terminal:
conda activate protein_modeling
# only if you did Module 7:
conda activate meta_env
```

- Both work first time? You are ready.
- Stuck? Every step is in the Setup Instructions PDF, Modules 1 to 7.
- **Next, 02 The Datasets:** the files we work on, and where they come from.
- **Then, 03 Basics of Protein Modelling.**